UNIVERSITÉ ALIOUNE DIOP DE BAMBEY

UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle

Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

Ecole-SN

Intelligence Éducative & Prédiction du Décrochage Scolaire

Éducation & Capital Humain

Durée	4 semaines
Équipe	2 étudiants
Domaine	Éducation & Capital Humain
Standard	M2 Approche Industrie | New Deal Technologique Sénégal
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • MLlib • Privacy by Design

1. Contexte et Objectifs

Le Sénégal affiche un taux de décrochage scolaire de ~35% au cycle moyen. Le taux de réussite au BFEM varie de 28% (régions rurales) à 72% (Dakar). Ce projet construit un système prédictif d'identification précoce des élèves à risque de décrochage, permettant aux inspections académiques de prioriser leurs interventions.

> ℹ️ **New Deal Technologique Sénégal**
> Ce projet s'inscrit dans la Stratégie Sénégal Numérique 2025-2035 et le Plan Sénégal Émergent. L'objectif est de former des ingénieurs Big Data capables de construire des solutions Data-Driven directement applicables aux défis de développement du Sénégal.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingestion multi-sources**	résultats DPRE, assiduité SIMEN, données socio-économiques ANSD
2	**Prédiction du risque de décrochage par élève (6 mois avant la fin d'année)**	Prédiction du risque de décrochage par élève (6 mois avant la fin d'année)
3	**Analyse des facteurs d'échec au BFEM/Bac par académie et type d'établissement**	Analyse des facteurs d'échec au BFEM/Bac par académie et type d'établissement
4	**Privacy by Design**	anonymisation des dossiers élèves (RGPD Sénégal + mineurs)
5	**Dashboard académies**	taux de réussite prédit, classes à risque, alertes directeurs

1.2 Architecture End-to-End

📄 **Architecture**

```
[ SOURCES ]  ──▶  [ NiFi ]  ──▶  [ KAFKA ]  ──▶  [ SPARK ML ]  ──▶  [ STORAGE ]
 DPRE résult.    Orchestration   Topic:           Prédiction          HBase/HDFS
 SIMEN assiduité & Routage       ecole_notes       Décrochage          Gold Layer
 ANSD socio-éco                  ecole_absences    Risque BFEM              |
                                                        |            [ ANALYTICS ]
                                                        ▼              Apache Hive
                                            [ ALERT LAYER ]        vue_academies
                                            Élève à risque         vue_bfem_bac
```

2. Prérequis & Infrastructure

ℹ️ Configuration minimale

RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 requis par Spark.

setup.sh

```
# Démarrage infrastructure
mkdir -p data dags models nifi_templates
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Démarrer le simulateur
python kafka_producer_ecole_sn.py &

# Interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

requirements.txt

```
# requirements.txt — Ecole-SN
pyspark==3.3.2
kafka-python==2.0.2
pandera==0.17.2
pandas==2.0.3
apache-airflow==2.7.3
mlflow==2.8.1
pyhive==0.7.0
happybase==1.2.0
matplotlib==3.7.3
seaborn==0.12.2
faker==30.1.0
scipy==1.11.4
numpy==1.24.4
```

3. Codes Sources Complets & Fonctionnels

3.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Crée les 3 tables HBase de Ecole-SN. À exécuter avant le démarrage du pipeline.

📄 **hbase_setup.py**

```
# hbase_setup.py — Tables HBase Ecole-SN
import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_ecole_sn_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        b'ecole:eleves_risque': {'b'meta': {'max_versions': 1}},  # Élèves à risque
décrochage (anonymisés)
        b'ecole:alertes_directeur': {'b'meta': {'max_versions': 1}},  # Alertes par
établissement (TTL 30j)
        b'ecole:stats_academie': {'b'meta': {'max_versions': 1}},  # KPIs académie (taux
réussite prédit)
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_ecole_sn_tables()
```

3.2 Simulateur de Données — kafka_producer

> ℹ️ **Données simulées réalistes**
> Le simulateur génère des données calées sur les réalités de Éducation & Capital Humain au Sénégal. Les champs PII (noms, IDs) sont présents dans le simulateur et seront anonymisés par la Privacy Layer Spark.

📄 **kafka_producer_ecole_sn.py**

```
# kafka_producer_ecole.py — Simulateur notes + absences élèves
from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime
import numpy as np
from faker import Faker

fake = Faker('fr_FR')
random.seed(42); np.random.seed(42)
ACADEMIES  = ['DAKAR','THIES','KAOLACK','ZIGUINCHOR','SAINT_LOUIS','LOUGA','TAMBACOUNDA']
MATIERES   = ['MATHS','FRANCAIS','SCIENCES','HISTOIRE_GEO','ANGLAIS','EPS','SVT']
NIVEAUX    = ['6EME','5EME','4EME','3EME','2NDE','1ERE','TERMINALE']
TYPES_ETAB = ['PUBLIC_RURAL','PUBLIC_URBAIN','PRIVE_LAIQUE','FRANCO_ARABE']

# Facteurs de risque de décrochage par type d'établissement
```

```
RISK_FACTOR =
{'PUBLIC_RURAL':0.6,'PUBLIC_URBAIN':0.3,'PRIVE_LAIQUE':0.1,'FRANCO_ARABE':0.4}

producer = KafkaProducer(bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v, ensure_ascii=False).encode())

def gen_note():
    academie   = random.choice(ACADEMIES)
    type_etab  = random.choice(TYPES_ETAB)
    risk       = RISK_FACTOR[type_etab]
    # Élève à risque : notes plus basses
    note_base  = 6.5 if random.random() < risk else 12.0
    return {
        'eleve_id':   f'ELV_{uuid.uuid4().hex[:10].upper()}',  # Sera anonymisé
        'nom':        fake.last_name(),   # Sera supprimé
        'prenom':     fake.first_name(),  # Sera supprimé
        'academie':   academie,
        'type_etab':  type_etab,
        'niveau':     random.choice(NIVEAUX),
        'matiere':    random.choice(MATIERES),
        'note':       round(max(0, min(20, np.random.normal(note_base, 3))), 1),
        'trimestre':  random.choice([1, 2, 3]),
        'annee':      datetime.utcnow().year,
        'timestamp':  datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

def gen_absence():
    academie = random.choice(ACADEMIES)
    return {
        'eleve_id':   f'ELV_{uuid.uuid4().hex[:10].upper()}',
        'academie':   academie,
        'nb_absences_sem': random.choices([0,1,2,3,5,10],weights=[40,25,15,10,7,3])[0],
        'justifiees': random.choice([True, False]),
        'semaine':    datetime.utcnow().isocalendar()[1],
        'timestamp':  datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

if __name__ == '__main__':
    print('Simulateur Ecole-SN démarré...')
    while True:
        for _ in range(5): producer.send('ecole_notes_raw', gen_note())
        producer.send('ecole_absences_raw', gen_absence())
        producer.flush(); time.sleep(3)
```

3.3 Pipeline Spark Streaming — streaming

> ⚠️ **Privacy critique**
> Drop des champs PII après anonymisation SHA-256. Les données sensibles ne doivent jamais apparaître dans HBase ou Hive.

streaming_ecole_sn.py

```
# streaming_ecole.py — Pipeline Spark Streaming Ecole-SN
# Score de risque décrochage + détection classes à risque
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, avg, sum as spark_sum, count, when, coalesce, current_timestamp
)
from pyspark.sql.types import StructType, StructField, StringType, FloatType, IntegerType,
BooleanType

SALT = os.environ.get('ECOLE_SECRET_SALT','default_salt')
spark = SparkSession.builder.appName('Ecole_SN_Streaming') \
    .config('spark.sql.shuffle.partitions','4').getOrCreate()
spark.sparkContext.setLogLevel('WARN')

note_schema = StructType([
    StructField('eleve_id',  StringType(), True),
    StructField('nom',       StringType(), True),
    StructField('prenom',    StringType(), True),
    StructField('academie',  StringType(), True),
    StructField('type_etab', StringType(), True),
    StructField('niveau',    StringType(), True),
    StructField('matiere',   StringType(), True),
    StructField('note',      FloatType(),  True),
    StructField('trimestre', IntegerType(),True),
    StructField('timestamp', StringType(), True),
])

notes_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('subscribe','ecole_notes_raw').load()
    .select(from_json(col('value').cast('string'), note_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
    # Privacy renforcée — MINEURS : drop nom, prénom, conservation eleve_id_secure
uniquement
    .withColumn('eleve_id_secure', sha2(concat(col('eleve_id'),lit(SALT)),256))
    .drop('eleve_id','nom','prenom')
)

# Score moyen par académie et niveau (fenêtre 4 semaines)
risque_df = (notes_df
    .withWatermark('event_ts','7 days')
    .groupBy(window('event_ts','28 days','7 days'),'academie','type_etab','niveau')
    .agg(
        avg('note').alias('note_moy'),
        count('eleve_id_secure').alias('nb_eleves')
    )
    .withColumn('risque_decrochage',
        when(coalesce(col('note_moy'),lit(10)) < 7.0,  lit('ELEVE'))
        .when(coalesce(col('note_moy'),lit(10)) < 10.0, lit('MOYEN'))
        .otherwise(lit('FAIBLE')))
)

q1 = (risque_df.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('topic','ecole_alerts')
    .option('checkpointLocation','/tmp/ecole_risque_ckpt')
    .outputMode('update').start())
q1.awaitTermination()
```

3.4 Tables Hive & Vues Analytics — hive_setup.sql

Les vues Hive utilisent systématiquement COALESCE sur les colonnes nullables pour éviter les NULL silencieux dans les agrégations.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Ecole-SN
CREATE DATABASE IF NOT EXISTS ecole_sn COMMENT 'Éducation Sénégal — UADB 2025';
USE ecole_sn;

CREATE TABLE IF NOT EXISTS notes_anonymisees (
    eleve_id_secure STRING, academie STRING, type_etab STRING,
    niveau STRING, matiere STRING, note DOUBLE,
    trimestre INT, annee INT, ingestion_ts TIMESTAMP
) PARTITIONED BY (academie_part STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

CREATE TABLE IF NOT EXISTS resultats_bfem_bac (
    academie STRING, annee INT, type_examen STRING,
    nb_candidats INT, nb_admis INT, taux_reussite DOUBLE,
    taux_predit DOUBLE
) STORED AS ORC;

-- Vue : Taux de réussite par académie et type d'établissement
CREATE OR REPLACE VIEW vue_academies_performance AS
SELECT academie, type_etab, niveau,
    AVG(COALESCE(note,0))       AS note_moyenne,
    COUNT(eleve_id_secure)      AS nb_eleves,
    SUM(CASE WHEN note < 7  THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS pct_eleves_risque,
    SUM(CASE WHEN note < 10 THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS pct_sous_moyenne
FROM notes_anonymisees
WHERE annee = YEAR(CURRENT_DATE())
GROUP BY academie, type_etab, niveau
ORDER BY pct_eleves_risque DESC;

-- Vue : Classes nécessitant intervention urgente
CREATE OR REPLACE VIEW vue_classes_urgence AS
SELECT academie, type_etab, niveau,
    AVG(note) AS note_moy,
    COUNT(*)  AS nb_obs
FROM notes_anonymisees
WHERE annee = YEAR(CURRENT_DATE())
GROUP BY academie, type_etab, niveau
HAVING AVG(note) < 7.0
ORDER BY note_moy ASC;
```

3.5 DAG Airflow MLOps

Le BranchPythonOperator détecte automatiquement les dérives et déclenche le réentraînement du modèle si nécessaire. Les recommandations sont écrites dans HBase pour un accès temps réel.

dags/ecole_sn_dag.py

```
# dags/ecole_sn_dag.py — DAG Airflow Ecole-SN
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('ecole_dag')
default_args = {'owner':'seye_ahmed','retries':2,'retry_delay':timedelta(minutes=5)}
```

```
def check_decrochage(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cur  = conn.cursor()
    cur.execute('SELECT COUNT(*) FROM ecole_sn.vue_classes_urgence')
    n = cur.fetchone()[0]
    logger.info(f'Classes en urgence : {n}')
    return 'retrain_model' if n > 5 else 'generate_alerts'

def retrain_model(**ctx):
    subprocess.run(['spark-submit','--master','spark://spark-master:7077',
        '/opt/models/train_decrochage_model.py'],check=True,timeout=3600)

def generate_alerts(**ctx):
    import happybase
    from pyhive import hive
    from datetime import datetime
    conn_hbase = happybase.Connection('hbase',port=9090); conn_hbase.open()
    conn_hive  = hive.Connection(host='hive-metastore',port=10000)
    cur = conn_hive.cursor()
    cur.execute('SELECT academie,type_etab,niveau,note_moy FROM
ecole_sn.vue_classes_urgence LIMIT 50')
    table = conn_hbase.table(b'ecole:alertes_directeur')
    for row in cur.fetchall():
        acad,etab,niv,note = row
        key = f'{acad}_{etab}_{niv}'.encode()
        table.put(key,{b'alerte:academie':acad.encode(),
                       b'alerte:note_moy':str(round(note,2)).encode(),
                       b'alerte:ts':datetime.utcnow().isoformat().encode()})
    conn_hbase.close()

with DAG('ecole_sn_monitoring', default_args=default_args,
         schedule_interval='0 5 * * 1', start_date=days_ago(1), catchup=False,
         tags=['ecole-sn','education','decrochage']) as dag:
    start  = DummyOperator(task_id='start')
    end    = DummyOperator(task_id='end')
    branch = BranchPythonOperator(task_id='check_decrochage',
                                  python_callable=check_decrochage,provide_context=True)
    t_train= PythonOperator(task_id='retrain_model',
                            python_callable=retrain_model,provide_context=True)
    t_alert= PythonOperator(task_id='generate_alerts',
                            python_callable=generate_alerts,provide_context=True)
    start >> branch >> [t_train, t_alert]
    t_train >> t_alert >> end; t_alert >> end
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet (GitHub) + template NiFi. Données visibles dans HDFS, topic Kafka ecole_notes_raw actif.
2	**Modèle ML & MLOps**	Script d'entraînement + RMSE/R^2 documentés + DAG Airflow avec BranchPythonOperator fonctionnel.
3	**Catalogue Hive**	Tables créées, vues fonctionnelles avec au moins 1 alerte visible dans HBase en démonstration.
4	**Dashboard & Recommandations**	Dashboard graphique (Matplotlib) + recommandations concrètes contextualisées au Éducation & Capital Humain sénégalais.

5. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Chaque critère est évalué selon 4 niveaux de maîtrise. La démonstration live en soutenance est obligatoire. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Sources	3 pts	NiFi absent	1 source/3	2 sources + topics	DPRE+SIMEN+ANSD, 2 topics, messages bien formés
Validation & Privacy (Mineurs)	4 pts	Eleve_id en clair, aucune validation	Schéma partiel	SHA-256 sans drop nom/prénom	Pandera strict + drop(nom, prenom, matricule) + 0 PII dans HBase — protection renforcée mineurs
Streaming & Détection Décrochage	6 pts	Pas de Spark	Lecture seule	Fenêtres 4 semaines + seuil fixe absences	Score de risque composite (absences + notes + progression) + watermarks
Prédiction BFEM/Bac & MLOps	4 pts	Aucun modèle	Règles simples	RF entraîné sans DAG	RF + Gradient Boosting comparés + DAG mensuel + F1-score documenté
Dashboard Académies & Recommandations	3 pts	Aucun dashboard	KPIs vides	Vues Hive OK	Alerte directeur + classe à risque + recommandation soutien scolaire
TOTAL	**20 pts**				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + captures d'écran	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
Tests pytest sur les schémas Pandera et les UDFs	+0.5 pt
Démo live d'une alerte HBase sans erreur en soutenance	+0.5 pt

🎓 **Bon courage pour Ecole-SN !**

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel